# TCC805x MCU BSP

## API Specification

## for Universal Asynchronous Receiver / Transmitter

**Rev. 1.10 [A]**

**2021-01-22**

# TABLE OF CONTENTS

## Contents


1 Introduction ..... 3
2 Terms and Abbreviations ..... 3
3 Source Files ..... 4
3.1 Location of Source Files ..... 4
3.2 Simple Description of Source Files ..... 4
4 Constants ..... 5
4.1 UART Channel Values ..... 5
4.2 UART Communication Method ..... 6
4.3 UART Debug Setting Values ..... 6
4.4 UART Register Address ..... 7
4.5 UART Flag Register Field Values ..... 8
4.6 UART Line Control Register Field Values ..... 9
4.7 UART Control Register Field Values ..... 11
5 APIs ..... 12
5.1 UART_Init ..... 12
5.2 UART_Open ..... 13
5.3 UART_Close ..... 14
5.4 UART_Write ..... 15
5.5 UART_Read ..... 16
5.6 UART_PutChar ..... 17
5.7 UART_GetChar ..... 18
5.8 UART_GetData ..... 19
5.9 UART_SetLineControlReg ..... 20
5.10 UART_SetInterruptClearReg ..... 21
5.11 UART_SetErrorClearReg ..... 22
5.12 UART_GetReceiveStatusReg ..... 23
5.13 UART_GetRawInterruptStatusReg ..... 24
6 How to use UART Driver ..... 25
6.1 Initialize and Read/Write Data ..... 25
7 References ..... 26
8 Revision History ..... 27
Rev. 1.10: 2021-01-22 ..... 27
Rev. 1.00: 2020-11-12 ..... 27


## Figures


Figure 3.1 Location of Source Files ..... 4


## Tables


Table 2.1 Terms and Abbreviation ..... 3

# 1 INTRODUCTION

This document describes the universal asynchronous receiver/transmitter device driver (hereinafter referred to as UART driver) of TCC805x MCU BSP. For more detailed information of the universal asynchronous receiver/transmitter in the TCC805x MCU Sub-System, refer to Chapter 12 Universal Asynchronous Receiver/Transmitter (UART) of the *"PART 11-MICOM SUB-SYSTEM"* in the *"TCC805x Full Specification"*. [1]

# 2 TERMS AND ABBREVIATIONS

**Table 2.1 Terms and Abbreviation**

| | |
|---|---|
| **UART** | Universal Asynchronous Receiver/Transmitter |

# 3 SOURCE FILES

## 3.1 Location of Source Files

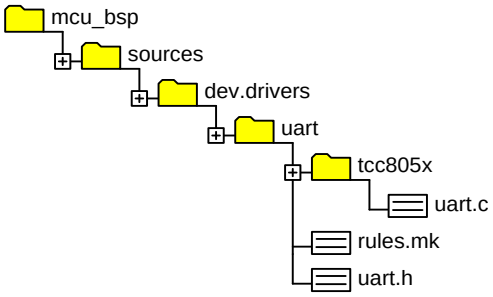


**Figure 3.1 Location of Source Files**

## 3.2 Simple Description of Source Files

- uart.c
  - UART driver source
- rules.mk
  - Rules used for build
- uart.h
  - UART driver API header

# 4 CONSTANTS

## 4.1 UART Channel Values

Constant values for UART channel.

### Declaration

```
#define UART_CH0      (0UL)
#define UART_CH1      (1UL)
#define UART_CH2      (2UL)
#define UART_CH3      (3UL)
#define UART_CH4      (4UL)
#define UART_CH5      (5UL)
#define UART_CH_MAX   (6UL)
```

### Description

| Value | Description |
|---|---|
| UART_CH0 | 1st UART Channel |
| UART_CH1 | 2nd UART Channel |
| UART_CH2 | 3rd UART Channel |
| UART_CH3 | 4th UART Channel |
| UART_CH4 | 5th UART Channel |
| UART_CH5 | 6th UART Channel |
| UART_CH_MAX | The maximum number of UART channel is 6. |

### Remark

None

## 4.2 UART Communication Method

Constant values for UART communication method

### Declaration

```
#define UART_POLLING_MODE    (0UL)
#define UART_INTR_MODE       (1UL)
#define UART_DMA_MODE        (2UL)
```

### Description

| Value | Description |
|---|---|
| UART_POLLING_MODE | The value can be read and written in the UART data register (DR). |
| UART_INTR_MODE | IRQ Interrupt is generated when reading or writing UART data. |
| UART_DMA_MODE | UART DMA is generated when reading or writing UART data. |

### Remark

None

## 4.3 UART Debug Setting Values

Constant values of UART channel for debugging

### Declaration

```
#define UART_DEBUG_CH        (UART_CH0)
#define UART_DEBUG_CLK       (48000000) // 48MHz
```

### Description

| Value | Description |
|---|---|
| UART_DEBUG_CH | UART channel used for debugging |
| UART_DEBUG_CLK | UART clock for debugging |

### Remark

None

## 4.4 UART Register Address

Offset values for TCC805x UART register address

### Declaration

```
#define UART_REG_DR      (0x00UL)
#define UART_REG_RSR     (0x04UL)
#define UART_REG_ECR     (0x04UL)
#define UART_REG_FR      (0x18UL)
#define UART_REG_ILPR    (0x20UL)
#define UART_REG_IBRD    (0x24UL)
#define UART_REG_FBRD    (0x28UL)
#define UART_REG_LCRH    (0x2cUL)
#define UART_REG_CR      (0x30UL)
#define UART_REG_IFLS    (0x34UL)
#define UART_REG_IMSC    (0x38UL)
#define UART_REG_RIS     (0x3cUL)
#define UART_REG_MIS     (0x40UL)
#define UART_REG_ICR     (0x44UL)
#define UART_REG_DMACR   (0x48UL)
```

### Description

| Value | Description |
|---|---|
| UART_REG_DR | Data register |
| UART_REG_RSR | Receive Status register |
| UART_REG_ECR | Error Clear register |
| UART_REG_FR | Flag register |
| UART_REG_ILPR | IrDA Low-power Counter register |
| UART_REG_IBRD | Integer Baud rate register |
| UART_REG_FBRD | Fractional Baud rate register |
| UART_REG_LCRH | Line Control register |
| UART_REG_CR | Control register |
| UART_REG_IFLS | Interrupt FIFO Level status register |
| UART_REG_IMSC | Interrupt Mask Set/Clear register |
| UART_REG_RIS | Raw Interrupt Status register |
| UART_REG_MIS | Masked Interrupt Status register |
| UART_REG_ICR | Interrupt Clear register |
| UART_REG_DMACR | DMA Control register |

### Remark

None

## 4.5 UART Flag Register Field Values

Field values of TCC805x UART Flag register

### Declaration

```
#define UART_FR_TXFE  (1UL << 7)
#define UART_FR_RXFF  (1UL << 6)
#define UART_FR_TXFF  (1UL << 5)
#define UART_FR_RXFE  (1UL << 4)
#define UART_FR_BUSY  (1UL << 3)
#define UART_FR_CTS   (1UL << 3)
```

### Description

| Value | Description |
|---|---|
| UART_FR_TXFE | Transmit FIFO empty<br>If the FIFO is disabled, this value is set when the transmit holding register is empty.<br>IF the FIFO is enabled, this value is set when the transmit FIFO is empty. |
| UART_FR_RXFF | Receive FIFO full<br>If the FIFO is disabled, this value is set when the receive holding register is full.<br>If the FIFO is enabled, this value is set when the receive FIFO is full. |
| UART_FR_TXFF | Transmit FIFO full<br>If the FIFO is disabled, this value is set when the transmit holding register is full.<br>If the FIFO is enabled, this value is set when the transmit FIFO is full. |
| UART_FR_RXFE | Receive FIFO empty<br>If the FIFO is disabled, this value is set when the receive holding register is empty.<br>If the FIFO is enabled, this value is set when the receive FIFO is empty. |
| UART_FR_BUSY | UART busy<br>If this value is set to 1, the UART is busy transmitting data. |
| UART_FR_CTS | Clear to send<br>This value is the complement of the UART clear to send. |

### Remark

None

# 4.6 UART Line Control Register Field Values

Field values of TCC805x UART Line Control Register

## Declaration

```
#define UART_LCRH_SPS        (1UL << 7)
#define UART_LCRH_WLEN(x)    ((x) << 5)
#define UART_LCRH_FEN        (1UL << 4)
#define UART_LCRH_STP2       (1UL << 3)
#define UART_LCRH_EPS        (1UL << 2)
#define UART_LCRH_PEN        (1UL << 1)
#define UART_LCRH_BRK        (1UL << 0)
```

## Description

| Value | Description |
|---|---|
| UART_LCRH_SPS | Select Stick parity<br>0: Stick parity is disabled<br>1: Either<br>-If the EPS bit is 0, the parity bit is transmitted and checked as 1.<br>-If the EPS bit is 1, the parity bit is transmitted and checked as 0. |
| UART_LCRH_WLEN | Word length<br>These values indicate the number of data bits transmitted or received in a frame as follows:<br><br>Input(x) is as follows:<br>`enum`<br>`{`<br>`  WORD_LEN_5 = 0,`<br>`  WORD_LEN_6,`<br>`  WORD_LEN_7,`<br>`  WORD_LEN_8`<br>`};` |
| UART_LCRH_FEN | Enable FIFOs<br>0: FIFOs are disabled (character mode) that is, the FIFOs become 1-byte-deep holding register.<br>1: Transmit and receive FIFO buffers are enabled. (FIFO mode). |
| UART_LCRH_STP2 | Select Two stop bits<br>If this value is set to 1, two stop bits are transmitted at the end of the frame.<br>The receive logic does not check for two stop bits being received. |
| UART_LCRH_EPS | Select Even parity<br>Controls the type of parity the UART uses during transmission and reception:<br><br>`enum`<br>`{`<br>`  PARITY_SPACE = 0,`<br>`  PARITY_EVEN,`<br>`  PARITY_ODD,` |

| Value | Description |
|---|---|
| | PARITY_MARK<br>}; |
| UART_LCRH_PEN | Enable Parity<br>0: Parity is disabled and no parity bit added to the data frame<br>1: Parity checking and generation is enabled. |
| UART_LCRH_BRK | Send break<br>If this value is 1, a low-level is continually output on the TXD output, after completing transmission of the current character.<br>For normal use, this value must be cleared to 0. |

## Remark

None

## 4.7 UART Control Register Field Values

Field values of TCC805x UART Control register

### Declaration

```
#define UART_CR_CTSEN     (1UL << 15)
#define UART_CR_RTSEN     (1UL << 14)
#define UART_CR_RTS       (1UL << 11)
#define UART_CR_RXE       (1UL << 9)
#define UART_CR_TXE       (1UL << 8)
#define UART_CR_LBE       (1UL << 7)
#define UART_CR_EN        (1UL << 0)
```

### Description

| Value | Description |
|---|---|
| UART_CR_CTSEN | Enable CTS hardware flow control<br>If this value is set to 1, CTS hardware flow control is enabled.<br>Data is only transmitted when the nUARTCTS signal is asserted. |
| UART_CR_RTSEN | Enable RTS hardware flow control<br>If this value is set to 1, RTS hardware flow control is enabled.<br>Data is only requested when there is space in the receive FIFO for it to be received. |
| UART_CR_RTS | Send Request<br>This value is the complement of the UART request to send, nRTS, modem status output. That is, when the bit is set to 1 then nRTS is LOW. |
| UART_CR_RXE | Enable receive<br>If this value is set to 1, the receive section of the UART is enabled. |
| UART_CR_TXE | Enable Transmit<br>If this value is set to 1, the transmit section of the UART is enabled. |
| UART_CR_LBE | Enable Loopback<br>If this value is set to 1, the UART TXD path is fed through to the UART RXD path. |
| UART_CR_EN | Enable UART<br>0: UART is disabled.<br>1: UART is enabled. |

### Remark

None

# 5 APIs

## 5.1 UART_Init

Initializes the UART Channel for debugging.

### Declaration

```
sint32 UART_Init
(
    void
);
```

### Parameters

None

### Return

None

### Remark

None

## 5.2 UART_Open

Sets the UART channel and options.

### Declaration

```
sint32 UART_Probe
(
    Uint8       ucCh,
    uint32      uiPriority,
    uint32      uiBaudrate,
    uint8       ucMode,
    uint8       ucCtsRts,
    uint8       ucPortCfg,
    uint8       ucWordLength,
    int8        ucFIFO,
    uint8       uc2StopBit,
    uint8       ucParity,
    GICIsrFunc fnCallback
);
```

### Parameters

| Value | Description |
|---|---|
| ucCh | [in] UART channel |
| uiPriority | [in] UART Interrupt priority (default value is 10) |
| uiBaudrate | [in] UART baud rate |
| ucMode | [in] UART mode (Polling/Interrupt/DMA) |
| ucCtsRts | [in] On/Off for CTS and RTS |
| ucPortCfg | [in] UART Port index corresponding to GPIO Multiport in the board_serial[] table |
| ucWordLength | [in] word length |
| ucFIFO | [in] on/off for FIFO |
| uc2StopBit | [in] on/off for 2 stop bit |
| ucParity | [in] parity bit (mask/even/odd/space) |
| fnCallback | [in] Callback function called when an interrupt occurs |

### Return

| Value | Description |
|---|---|
| 0 | Success |
| -1 | Failure |

### Remark

The UART port table (board_serial[] in uart.c) consists of all possible port combinations that can use UART ports. The port index corresponding to the UART channel should be recorded in the *UART_CHx_PORT_CFG* value.

## 5.3 UART_Close

Closes the UART Channel.

### Declaration

```
sint32 UART_Close
(
    Uint8       ucCh
);
```

### Parameters

| *Value* | *Description* |
|---|---|
| *ucCh* | *[in] UART channel* |

### Return

None

### Remark

None

## 5.4 UART_Write

Writes data to UART data register as much as the size.

### Declaration

```
sint32 UART_Write
(
    uint32         uiCh,
    const uint8 *  pucBuf,
    uint32         uiSize
);
```

### Parameters

| Value | Description |
|---|---|
| uiCh | [in] UART channel |
| pucBuf | [in] Transmit buffer |
| uiSize | [in] Size |

### Return

| Value | Description |
|---|---|
| 0 | Success |
| -1 | Failure |

### Remark

None

## 5.5 UART_Read

Receives data as much as the size from the buffer.

### Declaration

```
sint32 UART_Read
(
    uint32     uiCh,
    uint8 *    pucBuf,
    uint32     uiSize
);
```

### Parameters

| Value | Description |
|---|---|
| uiCh | [in] UART channel |
| pucBuf | [in] Receive buffer |
| uiSize | [in] Size |

### Return

| Value | Description |
|---|---|
| Positive number | Data size |
| -1 | Failure |

### Remark

None

## 5.6 UART_PutChar

Puts character data in UART data register.

### Declaration

```
sint32 UART_PutChar
(
    uint32      uiCh,
    uint8       ucChar
);
```

### Parameters

| Value | Description |
|---|---|
| uiCh | [in] UART channel |
| ucChar | [in] Character data |

### Return

| Value | Description |
|---|---|
| 0 | Success |
| -1 | Failure |

### Remark

None

## 5.7 UART_GetChar

Gets character data from UART data register.

### Declaration

```
sint32 UART_GetChar
(
    uint32     uiCh,
    sint32     iWait,
    sint8 *    pcErr
);
```

### Parameters

| Value | Description |
|---|---|
| uiCh | [in] UART channel |
| iWait | [in] Enable/Disable wait field |
| pcErr | [out] Error |

### Return

| Value | Description |
|---|---|
| Positive number | Received data |
| -1 | Failure |

### Remark

If the ***iWait*** flag is 1, UART_GetChar waits until the Receive FIFO becomes empty, and then gets data from the Data Register. If the ***iWait*** flag is 0, the UART_GetChar gets the data from the Data Register without a routine of waiting.

## 5.8 UART_GetData

Gets data from UART data register.

### Declaration

```
uint32 UART_GetData
(
    uint32     uiCh,
    sint32     iWait,
    sint8 *    pcErr
);
```

### Parameters

| Value | Description |
|---|---|
| uiCh | [in] UART channel |
| iWait | [in] Enable/Disable wait field |
| pcErr | [out] Error |

### Return

| Value | Description |
|---|---|
| Positive number | Received data |

### Remark

If the ***iWait*** flag is 1, UART_GetData waits until the Receive FIFO becomes empty, and then gets data from the Data Register. If the ***iWait*** flag is 0, the UART_GetData gets the data from Data Register without a routine of waiting.

## 5.9 UART_SetLineControlReg

Sets the UART Line Control Register.

### Declaration

```
void UART_SetLineControlReg
(
    uint32      ucCh,
    uint32      uiBits,
    uint8       uiEnabled
);
```

### Parameters

| Value | Description |
|---|---|
| ucCh | [in] UART channel |
| uiBits | [in] Bit field |
| uiEnabled | [in] Enable/Disable bit field |

### Return

None

### Remark

None

## 5.10 UART_SetInterruptClearReg

Clears the corresponding Interrupt.

### Declaration

```
Void UART_SetInterruptClearReg
(
    Uint8       ucCh,
    uint32      uiSetValue
);
```

### Parameters

| Value | Description |
|---|---|
| ucCh | [in] UART channel |
| uiSetValue | [in] value |

### Return

None

### Remark

None

## 5.11 UART_SetErrorClearReg

Clears the corresponding Error Interrupt.

### Declaration

```
Void UART_SetInterruptClearReg
(
    Uint8       ucCh,
    uint32      uiSetValue
);
```

### Parameters

| Value | Description |
|---|---|
| ucCh | [in] UART channel |
| uiSetValue | [in] value |

### Return

None

### Remark

None

## 5.12 UART_GetReceiveStatusReg

Gets the status value of the corresponding interrupt.

### Declaration

```
Void UART_GetReceiveStatusReg
(
    Uint8       ucCh
);
```

### Parameters

| Value | Description |
|---|---|
| ucCh | [in] UART channel |

### Return

| Value | Description |
|---|---|
| Positive number | Masked status value |

### Remark

None

## 5.13 UART_GetRawInterruptStatusReg

Gets the current raw status value of the corresponding interrupt.

### Declaration

```
Void UART_GetReceiveStatusReg
(
    Uint8      ucCh
);
```

### Parameters

| Value | Description |
|---|---|
| ucCh | [in] UART channel |

### Return

| Value | Description |
|---|---|
| Positive number | Raw status value |

### Remark

None

# 6 HOW TO USE UART DRIVER

UART driver must use ***UART_Init()*** function to set UART channel, baud rate, and mode. When reading data, ***UART_GetChar()*** or ***UART_Read()*** function is used. When writing data, ***UART_Write()*** function is used.

## 6.1 Initialize and Read/Write Data

```
// default settings

// Default Settings for uart channel 0
#define UART_DEBUG_CH     (UART_CH0)
#define UART_DEBUG_CLK        (48000000) // 48MHz
```

```
int main()
{
    sint8  error;
    sint32 ret    = 0;
    uint8  c      = 0;

    UART_Init(); //UART initialization

    for( ; ; )
    {
        ret = UART_GetChar(UART_CHANNEL0, Disable, (sint8 *)&error); //Read Character data

        if(ret > 0)
        {
            c   = ((uint8)ret & 0xFFUL);
            UART_Write(UART_CHANNEL0, &c, 1UL) //Write 1byte character data
        }
    }

    Return 0;
}

void UART_Init (void)
{
    (void)UART_Open(UART_CH0, GIC_PRIORITY_NO_MEAN, 115200UL, UART_POLLING_MODE, UART_CTSRTS_OFF,
                    35U, WORD_LEN_8, DISABLE_FIFO, TWO_STOP_BIT_OFF, PARITY_SPACE, NULL_PTR);
}
```

# 7 REFERENCES

[1] TCC805x Full Specification

[2] Contact Telechips for more details: sales@telechips.com

# 8 REVISION HISTORY

## Rev. 1.10: 2021-01-22

- Updated
  - Chapter 4.3: Declaration and description
  - Chapter 4.5, Chapter 4.6, Chapter 4.7: Description
  - Chapter 5.1: Return
  - Chapter 5.4: UART_Write
  - Chapter 5.5: UART_Read
  - Chapter 6.1: Source code

- Added
  - Chapter 5.2: UART_Open
  - Chapter 5.3: UART_Close
  - Chapter 5.9: UART_SetLineControlReg
  - Chapter 5.10: UART_SetInterruptClearReg
  - Chapter 5.11: UART_SetErrorClearReg
  - Chapter 5.12: UART_GetReceiveStatusReg
  - Chapter 5.13: UART_GetRawInterruptStatusReg

- Deleted
  - Chapter 4.3: UART Channel Usage.
  - Chapter 4.5: UART Channel Setting Values.
  - Chapter 5.1: UART_SetLineControlReg
  - Chapter 5.2: UART_DecideGpioPort
  - Chapter 5.4: UART_Probe

## Rev. 1.00: 2020-11-12

- First version release

# DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.
No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.
This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.
Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

# COPYRIGHT STATEMENT



# Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

**For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:**
"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit http://mp3licensing.com".

**For customers who use other firmware of mp3:**
"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit http://mp3licensing.com".

**For customers who use Digital Wave DRA solution:**
"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

**For customers who use DTS technology:**
"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

**For customers who use Dolby technology:**
"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

**For customers who use Google technology:**
"Copyright © 2013 Google Inc. All rights reserved."

**For customers who use MS technology:**
"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."